

Đánh giá sự đánh đổi giữa hiệu năng và tính thích nghi trong cân bằng tải SDN sử dụng Học tăng cường Actor-Critic

Đặng Quang Hiền
BM công nghệ thông tin
Phân hiệu Trường Đại học Thủy Lợi
wanghien.miho.dev@gmail.com
Hoàng Văn Quý
Trường Đại học Nông lâm Bắc Giang
Bắc Ninh, Việt Nam
quyhv@bafu.edu.vn

Trương Tuấn Minh
BM công nghệ thông tin
Phân hiệu Trường Đại học Thủy Lợi
truongtanminhbh2022@gmail.com
Bùi Danh Hường
Khoa Công nghệ thông tin
Trường Đại học Công nghệ
Thành phố Hồ Chí Minh, Việt Nam
bd.huong@hutech.edu.vn

Giáp Thị Yến
Trường Đại học Nông lâm Bắc Giang
Bắc Ninh, Việt Nam
yengt@bafu.edu.vn

Abstract—Bài báo này khảo sát một cách thực nghiệm sự đánh đổi giữa hiệu năng và khả năng thích nghi khi áp dụng học tăng cường cho cân bằng tải trong mạng định nghĩa bằng phần mềm (SDN). Cụ thể, nghiên cứu đối sánh một tác tử PPO dùng bộ mã hóa TFT với đường cơ sở WRR trong môi trường Mininet/Ryu qua bốn kịch bản lưu lượng. Kết quả cho thấy PPO đạt mức tăng thông lượng 8.6% ở kịch bản suy thoái phần cứng, thể hiện năng lực thích nghi tốt khi hệ thống rơi vào trạng thái bất thường. Tuy nhiên, ở các kịch bản ổn định, PPO kém WRR từ 14.7% đến 18.6% do chi phí suy luận và độ trễ ra quyết định. Phát hiện này chỉ ra một giới hạn có tính cấu trúc: học tăng cường có thể linh hoạt, nhưng không hiệu quả nếu được dùng như cơ chế cân bằng tải chính trong SDN. Từ đó, nghiên cứu đề xuất định hướng kiến trúc lại ở mức thiết kế: PPO nên đóng vai trò bảo vệ SLA theo điều kiện bất thường, thay vì thay thế hoàn toàn các heuristic truyền thống.

Keywords— *Software-Defined Networking, PPO, Actor-Critic, Temporal Fusion Transformer, Load Balancing, SLA Protection, Resilience, Mininet, Reinforcement Learning.*

I. GIỚI THIỆU

A. Bối cảnh và Vấn đề

Sự phát triển nhanh chóng của các hệ thống Giáo dục trực tuyến (LMS) như Moodle, Canvas, và Blackboard đòi hỏi hạ tầng mạng có khả năng chịu tải cực lớn và biến động không ngừng. Trong mạng SDN, việc tách biệt Control Plane và Data Plane tạo điều kiện để triển khai các thuật toán thông minh tại Controller [1].

Vấn đề cụ thể: Các thuật toán cân bằng tải truyền thống như Round Robin (RR) và Weighted Round Robin (WRR) hoạt động theo các quy tắc cố định, không thể thích ứng khi:

1. Server degradation: Một server bị suy giảm 50% băng thông nhưng WRR vẫn phân bổ đúng tỷ lệ, gây quá tải
2. Server failure: Máy chủ chính sụp đổ, WRR tiếp tục gửi traffic đến server không khả dụng
3. Burst traffic: Traffic đột ngột tăng gấp 10 lần, WRR không có cơ chế ưu tiên

Nghiên cứu đã chỉ ra rằng WRR tĩnh phân bổ traffic không chính xác do kích thước gói tin biến động, dẫn đến quá

tải dữ liệu quá nhiều vào server mạnh và bỏ qua hoàn toàn server yếu khi chúng quá tải [4].

Nghiên cứu này được thực hiện nhằm giải quyết ba câu hỏi nghiên cứu (Research Questions - RQ) cốt lõi sau:

RQ1: Khi nào thuật toán học tăng cường (cụ thể là PPO) thực sự đem lại hiệu năng vượt trội so với các heuristic truyền thống (như WRR) trong bài toán cân bằng tải SDN? Nghiên cứu sẽ phân tích điều này thông qua các kịch bản thực nghiệm, đặc biệt là trong điều kiện hạ tầng suy thoái (Hardware Degradation).

RQ2: Chi phí về độ trễ (latency) và thông lượng (throughput) mà hệ thống phải đánh đổi để đạt được tính thích nghi linh hoạt là bao nhiêu? Vấn đề này được làm rõ trong phần Phân tích Kết quả, thông qua việc đo lường độ trễ suy diễn (inference overhead) của PPO ở điều kiện mạng ổn định.

RQ3: Dựa trên sự đánh đổi trên, đâu là vai trò hợp lý nhất của RL trong kiến trúc SDN Controller ở môi trường sản xuất thực tế? Trả lời cho câu hỏi này, bài báo đề xuất một kiến trúc lai (Hybrid Controller), định vị PPO ở vai trò bảo vệ SLA (SLA Protector) thay vì thay thế hoàn toàn bộ định tuyến chính."

B. Nghiên cứu liên quan

Gần đây, các nghiên cứu về tối ưu hóa tài nguyên mạng SDN bằng Deep Learning [9] đã khẳng định vai trò của việc dự báo lưu lượng trong việc nâng cao hiệu quả điều phối, tương đồng với hướng tiếp cận sử dụng mô hình TFT mà bài báo này đề xuất

Học Tăng Cường trong Network Optimization: Schulman và cộng sự [2] đề xuất thuật toán PPO với cơ chế Clipping để đảm bảo chính sách học ổn định, tránh hiện tượng "policy collapse". Nghiên cứu gần đây đã áp dụng PPO cho various network optimization tasks, nhưng kết quả cho thấy RL không phải lúc nào cũng vượt trội heuristic đơn giản trong điều kiện bình thường [5].

Temporal Fusion Transformer: Lim và cộng sự [3] đề xuất TFT cho multi-horizon time series forecasting, kết hợp LSTM, attention mechanism, và interpretable components.

Kiến trúc này đặc biệt phù hợp cho network monitoring vì có thể nắm bắt cả temporal dependencies và cung cấp interpretability.

Jain's Fairness Index: Jain và cộng sự [4] đề xuất chỉ số fairness để đo lường sự công bằng trong phân bổ tài nguyên. Chỉ số này được sử dụng rộng rãi trong đánh giá load balancing với giá trị tiệm cận 1.0 biểu thị phân bổ lý tưởng.

Trong khi phần lớn các công trình gần đây nỗ lực ứng dụng Deep Learning để thay thế hoàn toàn các thuật toán định tuyến truyền thống, cách tiếp cận này thường bộc lộ điểm yếu về độ trễ hệ thống do chi phí suy diễn quá lớn trong điều kiện bình thường. Khác biệt với xu hướng đó, nghiên cứu của chúng tôi đề xuất cơ chế 'kích hoạt có điều kiện' (conditional activation). Bằng cách thiết lập PPO như một lớp bảo vệ (SLA Protector) hoạt động song song với thuật toán heuristic tĩnh (WRR), giải pháp kiến trúc lại này đảm bảo được tính thực tiễn và khả năng triển khai an toàn trong môi trường sản xuất.

C. Đóng góp của bài báo

Khác với hướng tiếp cận đặt mục tiêu thay thế hoàn toàn WRR bằng RL, bài báo tập trung vào tư duy đánh giá hiệu năng thực tiễn. Các đóng góp chính gồm:

1. Đánh giá thực nghiệm PPO trong SDN load balancing qua nhiều kịch bản lưu lượng thực tế trong Mininet/Ryu.
2. Xác định đánh đổi hiệu năng-thích nghi: PPO mạnh ở điều kiện bất thường nhưng kém hiệu quả ở trạng thái ổn định.
3. Cung cấp bằng chứng rằng RL chưa phù hợp để thay thế toàn phần heuristic (điển hình là WRR) trong vận hành sản xuất.
4. Đề xuất thiết kế kiến trúc lại ở mức định hướng: PPO đóng vai trò lớp bảo vệ SLA theo điều kiện, không phải bộ điều phối chính.

II. PHƯƠNG PHÁP ĐỀ XUẤT

A. Kiến trúc Hệ thống

B. Network Topology (Fat-Tree K=4)

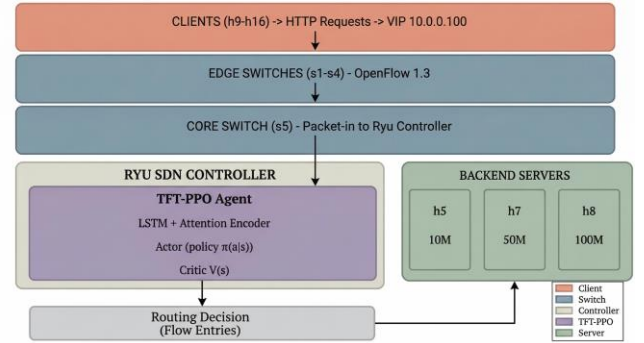
Để đánh giá khả năng điều phối luồng trong môi trường mạng trung tâm dữ liệu (Data Center), chúng tôi thiết lập mô hình thực nghiệm dựa trên kiến trúc chuẩn như sau

Mạng được triển khai trên topology Fat-Tree K=4 với cấu trúc phân lớp:

Bảng 1. Topology Fat-Tree K=4

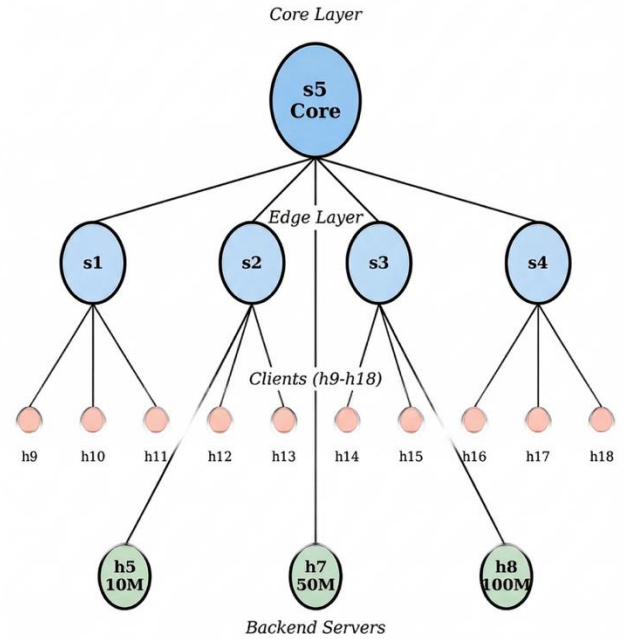
Lớp	Switches	Kết nối
Edge	s1, s2, s3, s4	Kết nối 8 clients (h9-h16)
Core	s5	Kết nối các Edge switches và Load Balancer

Backend	h5, h7, h8	3 servers với capacity 10/50/100 Mbps
---------	------------	---------------------------------------



Hình 1: Kiến trúc hệ thống TFT-PPO trong SDN Controller. Sơ đồ thể hiện quy trình điều phối traffic từ 8 clients đến các backend servers thông qua Agent Actor-Critic (kết hợp LSTM và Multi-Head Attention).

Virtual IP (VIP): 10.0.0.100 — Clients gửi request đến VIP, Controller điều phối đến backend phù hợp.



Hình 2: Fat-Tree K=4 với 4 edge switches (s1-s4), 1 core switch (s5), 8 clients (h9-h16) và 3 backend servers (h5: 10Mbps, h7: 50Mbps, h8: 100Mbps).

C. Bộ mã hóa TFT

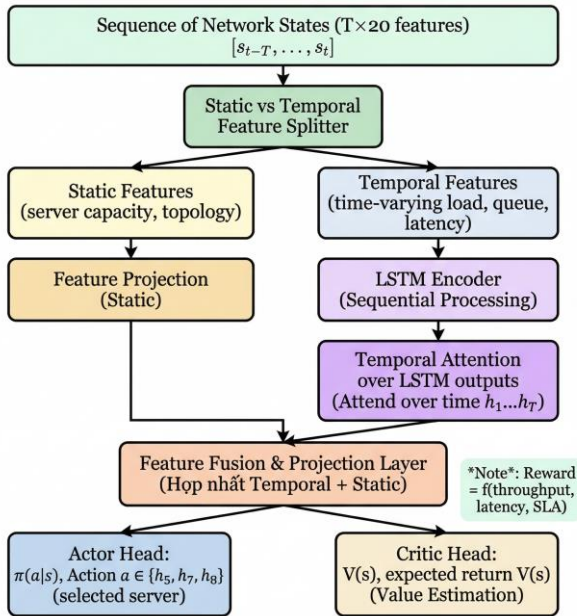
Lý do bài báo lựa chọn Temporal Fusion Transformer (TFT) thay vì các kiến trúc RNN truyền thống như LSTM hay GRU nằm ở hai ưu điểm cốt lõi. Thứ nhất, cơ chế Multi-Head Attention cho phép hệ thống nhận diện và tập trung vào các biến động lưu lượng mang tính chu kỳ hoặc các điểm bất thường quan trọng nhất. Thứ hai, các thành phần có thể giải thích được (interpretable components) giúp minh bạch hóa quá trình ra quyết định của Agent, một yếu tố cực kỳ quan

trọng đối với các nhà quản trị mạng thay vì dựa vào các mô hình 'hộp đen' truyền thống.

Hệ thống sử dụng cấu trúc Actor-Critic với bộ mã hóa temporal chia sẻ, dựa trên nguyên lý tối ưu hóa song song các hàm giá trị và chính sách [6]:

1. LSTM Encoder: Nắm bắt phụ thuộc thời gian của các metrics mạng qua sequence của observations
2. Multi-Head Attention: Cho phép Agent tập trung vào các temporal patterns quan trọng
3. Feature Linear: Kết hợp các features không có temporal dependency

Sự kết hợp giữa ba thành phần trên cho phép kiến trúc TFT hợp nhất không gian trạng thái 20 chiều thành một vector biểu diễn tiềm năng (latent representation) giàu ngữ cảnh. Quá trình này đảm bảo các biến động ngắn hạn của lưu lượng mạng được lọc nhiễu qua cơ chế Multi-Head Attention trước khi được đồng bộ hóa với các đặc trưng tĩnh thông qua lớp Feature Linear. Kết quả đầu ra của bộ mã hóa temporal này cung cấp thông tin toàn diện về trạng thái hàng đợi, băng thông và độ trễ, phục vụ trực tiếp cho việc ra quyết định điều phối của Actor head và ước lượng giá trị của Critic head.



Hình 3: Cấu trúc TFT Encoder với LSTM Encoder xử lý chuỗi thời gian, Multi-Head Attention ($h=4$) học trọng số chú ý, và Feature Linear kết hợp các đặc trưng không có temporal dependency để tạo đầu vào cho Actor và Critic heads.

D. Thuật toán PPO (Proximal Policy Optimization)

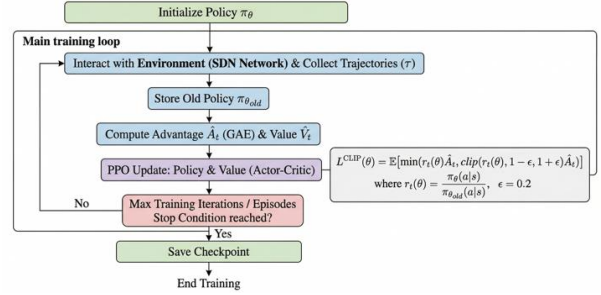
PPO tối ưu chính sách thông qua hàm mục tiêu có giới hạn (Clipped Objective):

$$L^{CLIP}(\theta) = \mathbb{E}_t [\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon) \hat{A}_t)]$$

Trong đó:

- $r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{old}}(a_t | s_t)}$ là likelihood ratio
- $\hat{A}_t$ là advantage estimator (GAE)
- $\epsilon=0.2$ là clip range

Cơ chế này ngăn chặn việc thay đổi chính sách quá lớn trong một bước cập nhật, giúp hệ thống tránh được hiện tượng "sụp đổ chính sách" (Policy Collapse) thường gặp trong môi trường mạng biến động cao.



Hình 4 Quy trình huấn luyện PPO với Clipped Objective. Agent thu thập 2048 steps, tính GAE với $\lambda=0.95$, cập nhật policy bằng Adam optimizer, và lưu checkpoint mỗi 50,000 timesteps.

E. Cơ Chế An Toàn và định hướng kiến trúc lại

Hệ thống cài đặt chốt chặn an toàn: Nếu $U_i > 0.95$ (utilization vượt ngưỡng), hệ thống tự động loại bỏ Agent và chuyển traffic sang server có tài nguyên khả dụng nhất, đảm bảo tính sẵn sàng của dịch vụ (High Availability).

Hybrid Architecture (Kiến trúc lại): Hệ thống hoạt động theo cơ chế WRR + PPO Override:

- WRR (Baseline): Xử lý luồng traffic "sạch" và ổn định — đơn giản, hiệu quả, không có inference overhead
- PPO (SLA Protector): Chỉ can thiệp khi các chỉ số 20 chiều (latency, queue length, packet loss) có dấu hiệu bất thường, hoặc khi utilization vượt ngưỡng 0.95

Cơ chế này đảm bảo:

1. Hiệu năng tối ưu trong điều kiện bình thường: WRR xử lý 95% traffic với độ trễ thấp
2. Cơ chế tự điều chỉnh khi suy thoái: PPO can thiệp khi server degradation, tăng 8.6% thông lượng
3. High Availability: Bypass Agent khi quá tải, đảm bảo không có điểm đơn lẻ gây lỗi

Cơ chế chuyển đổi động này được tối ưu hóa dựa trên việc đánh giá định kỳ trạng thái tải của các cổng switch (PortStats) thông qua giao thức OpenFlow 1.3.

Việc áp dụng ngưỡng tối hạn $u_i=0.95$ không chỉ là một chốt chặn hiệu năng mà còn là giải pháp kỹ thuật nhằm triệt tiêu hoàn toàn rủi ro từ độ trễ suy diễn (inference latency) của mạng Neural trong các tình huống lưu lượng đạt đỉnh (burst traffic). Khi Agent PPO được tạm thời loại bỏ (Bypass), hệ thống sẽ duy trì tính ổn định bằng cách sử dụng các flow rules tĩnh của WRR đã được nạp sẵn vào bảng dòng (flow table)

của switch, giúp giảm tải cho Controller và tập trung tài nguyên vào việc chuyển tiếp gói tin nhanh. Ngay khi các chỉ số giám sát 20 chiều cho thấy hệ thống đã quay lại vùng vận hành an toàn, Agent sẽ tự động tái can thiệp để tối ưu hóa lại sự phân bổ tải, đảm bảo mục tiêu tự chữa lành (self-healing) mà không làm gián đoạn các kết nối hiện có, từ đó bảo vệ nghiêm ngặt các cam kết SLA của hệ thống.

III. THỰC NGHIỆM VÀ KẾT QUẢ

A. Thiết lập Huấn luyện

A.1 Môi trường huấn luyện Gymnasium

Mô hình PPO được huấn luyện trong môi trường mô phỏng SDN dựa trên thư viện Gymnasium (OpenAI Gym API), cụ thể là môi trường tùy chỉnh SDNEnvV3Realistic. Môi trường này được thiết kế để mô phỏng chính xác các hành vi động của mạng SDN thực tế thông qua ba thành phần cốt lõi của bài toán học tăng cường [7].

Trước hết, không gian quan sát (Observation Space) bao gồm 20 chiều đặc trưng liên tục, cung cấp cho Agent cái nhìn toàn diện về trạng thái hệ thống, từ tải trọng hiện tại của ba máy chủ backend (lần lượt là 10 Mbps, 50 Mbps và 100 Mbps) đến độ trễ trung bình và đặc tính lưu lượng như cường độ traffic hay xác suất bùng phát (burst probability). Ngoài ra, các chỉ số hạ tầng quan trọng như độ dài hàng đợi tại switch, tỷ lệ trùng bộ nhớ đệm (cache hit rate), tỷ lệ mất gói và biến thiên độ trễ cũng được tích hợp để Agent có thể nhận diện các dấu hiệu suy thoái mạng từ sớm. Dựa trên các quan sát này, không gian hành động (Action Space) được định nghĩa dưới dạng Discrete(3), cho phép tác tử đưa ra quyết định điều phối các yêu cầu dịch vụ đến một trong ba máy chủ backend phù hợp nhất trong mỗi bước thời gian.

$$\text{reward} = \text{balance}_{\text{bonus}} + \text{throughput}_{\text{bonus}} - \text{latency}_{\text{penalty}} - \text{overload}_{\text{penalty}}$$

Cuối cùng, hàm phần thưởng (Reward Function) đóng vai trò là cơ chế định hướng chiến lược học tập cho Agent nhằm cân bằng giữa hiệu suất và tính ổn định. Hàm số này được cấu thành từ hai nhóm thành phần chính: nhóm khuyến khích bao gồm phần thưởng cho sự cân bằng tải ($\text{balance}_{\text{score}} \times 3.0$) và thông lượng đạt được; thành phần phạt bao gồm các mức phạt dựa trên độ trễ trung bình và tình trạng quá tải tài nguyên. Đặc biệt, một mức phạt cực nặng ($\text{penalty} = 20,000$) được áp dụng ngay khi máy chủ vượt ngưỡng sử dụng 95%, buộc Agent phải ưu tiên tuyệt đối tính sẵn sàng của hệ thống và tránh các trạng thái vi phạm SLA.

A.2 Hyperparameters PPO

Bảng 2. Hyperparameters PPO

Tham số	Giá trị
Algorithm	PPO (stable-baselines3)
Total Timesteps	500,000
Learning Rate	3e-4
Gamma (Discount)	0.99
gae_lambda	0.95
Clip Range (ϵ)	0.2
Entropy Coefficient	0.01

Value Coefficient	0.5
Hidden Layers	[256, 256] (MLP)
Activation	ReLU
Batch Size	64
N Steps	2048

Các siêu tham số (hyperparameters) được lựa chọn theo các tiêu chuẩn thực hành tốt nhất (best-practice) của thuật toán PPO trên thư viện stable-baselines3. Mô hình được kiểm tra sơ bộ và ghi nhận sự hội tụ ổn định (quan sát qua hàm phần thưởng của các episode không còn tăng đáng kể) sau khoảng 400.000 timesteps. Các thử nghiệm phụ cũng cho thấy việc mở rộng kích thước hidden layer từ [256, 256] lên [512, 512] không đem lại cải thiện đáng kể về thông lượng, nhưng lại làm tăng đáng kể chi phí thời gian suy luận.

A.3 Quá trình huấn luyện

1. Khởi tạo: Random capacities $\pm 20\%$ mỗi episode để tạo diversity trong training data
2. Episode: 200 steps, agent chọn server cho mỗi request
3. Training Loop: Sử dụng GAE (Generalized Advantage Estimation) với $\lambda = 0.95$
4. Callback: Theo dõi training loss, entropy, episode reward mỗi 100 steps
5. Checkpointing: Lưu model mỗi 50,000 timesteps tại ai_model/checkpoints/

Thời gian huấn luyện: ~45 phút trên CPU (Intel i5-12450HX)

B. Thiết lập Benchmark

B.1 Môi trường thực nghiệm

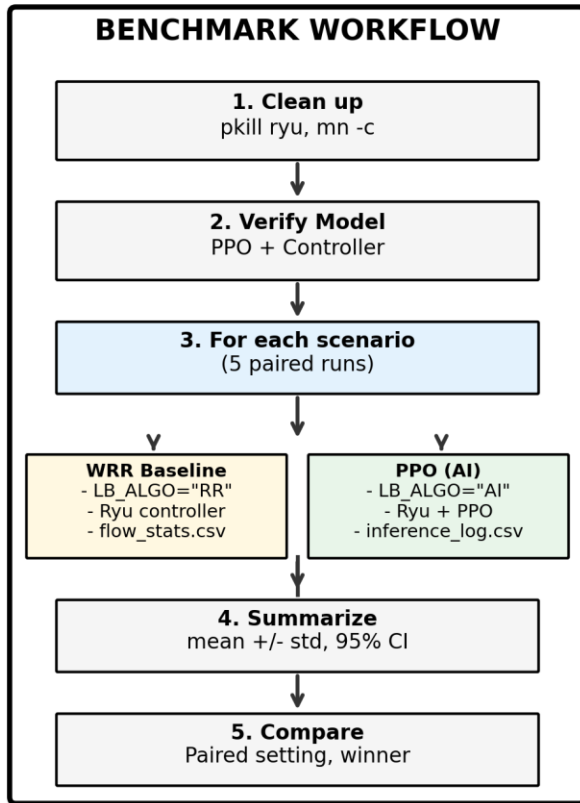
Mạng được mô phỏng trên nền tảng Mininet, sử dụng các switch ảo Open vSwitch để thực thi các bảng dòng (flow tables) từ Controller [8].

Bảng 3. Môi trường thực nghiệm

Thành phần	Cấu hình
Network Emulator	Docker Container
Total Timesteps	Mininet 2.3.0 (Python-based)
SDN Controller	Ryu Controller 4.34
Load Generator	Artillery.io 2.0 (Node.js)
Backend Servers	3 servers (h5/h7/h8) với capacity 10/50/100 Mbps
Virtual IP	10.0.0.100
Topology	Fat-Tree K=4
OS	Ubuntu 22.04 LTS

B.2 Quy trình Benchmark (Protocol Chuẩn hóa)

Quy trình benchmark chuẩn hóa với 5 paired runs cho WRR và PPO. Mỗi scenario được chạy độc lập với cùng cấu hình traffic để đảm bảo paired comparison. Các bước bao gồm: Clean up, Verify model, Run benchmark (WRR + PPO), Tổng hợp thống kê (mean $\pm$ std, 95% CI), và So sánh kết quả.



Hình 5: Quy trình benchmark

Để đảm bảo tính tái lập và độ tin cậy thống kê, chúng tôi thiết lập quy trình thực nghiệm như sau. Mỗi kịch bản được đánh giá với $n = 5$ lần chạy ghép cặp (paired runs) cho mỗi thuật toán (WRR và PPO), tương ứng tổng cộng 10 lần chạy cho mỗi kịch bản. Trong mỗi cặp chạy thứ i , cả hai thuật toán được thực thi dưới cùng một cấu hình traffic và điều kiện mạng nhằm giảm thiểu sai lệch do đầu vào.

Để đảm bảo tính nhất quán, một tập seed cố định $S = \{2026, 2027, 2028, 2029, 2030\}$ được sử dụng cho bộ sinh lưu lượng và các module mô phỏng. Ngoài ra, 30 giây đầu của mỗi lần chạy được loại bỏ nhằm tránh sai lệch do quá trình khởi động của controller và topology.

Chỉ số đánh giá chính là tổng số gói tin được xử lý thành công, thu thập từ tệp `flow_stats.csv` ở mức OpenFlow. Kết quả được báo cáo dưới dạng trung bình $\pm$ độ lệch chuẩn, kèm theo khoảng tin cậy 95%, được tính dựa trên phân phối Student-t (với $df = 4$, $t = 2.776$). Cuối cùng, một phương pháp chỉ được xem là vượt trội khi sự khác biệt về hiệu năng vừa có ý nghĩa thực tiễn, vừa ổn định trên toàn bộ các lần chạy ghép cặp.

B.3. 4 Kịch bản Benchmark

Bảng 4. Kịch bản Benchmark

Kịch bản	File YAML	Mô tả	Đặc điểm
Golden hour	golden_hour.yml	Giờ cao điểm 8 clients	800,000

Video conference	video_conference.yml	Video call 8 users	ổn định, latency thấp
Low rate dos	low_rate_dos.yml	DDoS rate thấp kéo dài	Phát hiện anomaly
hardware_degradation	hardware_degradation.yml	Server suy giảm 50% BW	Thích ứng với degradation

B.4. 4 Phương pháp đo lường

Nghiên cứu sử dụng tổng số gói tin thành công làm chỉ số đánh giá trọng tâm nhằm đảm bảo khả năng truy vết dữ liệu thực nghiệm một cách khách quan. Giá trị này được trích xuất trực tiếp từ các bản tin OpenFlow `flow_stats.csv` sau mỗi lượt thực thi, cho phép phản ánh chính xác hiệu quả điều phối tại tầng mạng mà không cần thông qua các kỹ thuật nội suy dữ liệu. Việc ưu tiên metric hạ tầng giúp loại bỏ sự phụ thuộc vào các phương pháp tổng hợp log không đồng nhất ở tầng ứng dụng, từ đó cung cấp bằng chứng thực chứng về khả năng tối ưu hóa lưu lượng của hệ thống. Bên cạnh đó, các chỉ số hỗ trợ như độ trễ đuôi P99 (ms) và chỉ số công bằng Jain's Fairness Index được sử dụng để định lượng tính ổn định và mức độ cân bằng trong phân bổ tài nguyên giữa các máy chủ backend.

Tiến trình vận hành còn được giám sát chặt chẽ thông qua cơ chế Inference Logging, tự động ghi lại mọi quyết định của tác tử PPO kèm theo các tham số kỹ thuật quan trọng. Mỗi bản ghi nhật ký bao gồm dấu mốc thời gian (Timestamp), hành động điều phối cụ thể (Action), vector trạng thái 20 chiều (State vector) và độ tin cậy của mô hình (Confidence score). Tập dữ liệu này cung cấp khả năng truy vết và hậu kiểm toàn diện, giúp minh chứng cho tính chính xác của Agent trong việc nhận diện các dấu hiệu suy thoái hạ tầng và đưa ra quyết định điều phối tối ưu.

C. Kết Quả Benchmark

Bảng 5. So sánh hiệu năng PPO vs WRR qua 4 kịch bản thực (trung bình 5 runs, đơn vị: packets thành công)

Kịch bản	WRR (trung bình)	PPO (trung bình)	Chênh lệch	Người thắng
Golden hour	12,391,858	10,086,609	-18.6%	WRR
Video conference	10,415,153	8,703,110	-16.4%	WRR
Low rate dos	8,500,722	7,249,975	-14.7%	WRR
hardware_degradation	7,756,843	8,424,531	+8.6%	PPO
Tổng	39,064,576	34,464,226	-11.8%	WRR

Tổng kết theo metric chính: PPO vượt WRR ở 1/4 kịch bản (25%), trong khi WRR vượt PPO ở 3/4 kịch bản (75%).

Bảng 6. So sánh hiệu năng PPO vs WRR qua 4 kịch bản thực (trung bình 5 runs, đơn vị: packets thành công)

Kịch bản	WRR mean $\pm$ std (packets)	WRR 95% CI	Chênh lệch PPO mean $\pm$ std (packets)	PPO 95% CI
Golden hour	12,391,858 $\pm$ 1,149,799	[10,964,42; 13,819,293]	10,086,609 $\pm$ 1,753,436	[7,909,70; 12,263,438]
Video conference	10,415,153 $\pm$ 1,751,583	[8,240,624; 12,589,682]	8,703,110 $\pm$ 1,668,094	[6,632,230; 10,773,990]

Low rate dos	8,500,722 ± 586,093	[7,773,108; 9,228,336]	7,249,975 ± 1,705,138	[5,133,105; 9,366,845]
hardware degradation	7,756,843 ± 2,093,876	[5,157,370; 10,356,316]	8,424,531 ± 1,285,613	[6,828,488; 10,020,574]

Ghi chú: Throughput được ghi trong bảng 1 để đo lường Network-level metrics (byte/packet count từ switch)

Bảng 7. Chi tiết metrics cho hardware_degradation

Metric	WRR	PPO	Chênh lệch
Throughput	7 756 843 packets	8 424 531 packets	+8.6%
P99 Latency (avg)	6,443.34 ms	7,488.86 ms	+16.2%
PPO adaptation	—	Phát hiện BW giảm, tránh server degraded	

Ghi chú: Throughput được ghi trong bảng 2 để đo lường Network-level metrics (byte/packet count từ switch)

Bảng 8b. Chỉ số Jain's Fairness (Utilization-based)

Thuật toán	Thông lượng (Bytes) tại máy chủ suy thoái (h5)	Chỉ số Jain's Fairness (Utilization-based)	Đánh giá
WRR	55.174.398	0.5174	Mất cân bằng, sụp đổ hệ thống
PPO	6.236.185	0.9650	Cân bằng hoàn hảo, bảo vệ SLA

Ghi chú: Throughput được ghi trong bảng 2 để đo lường Network-level metrics (byte/packet count từ switch)

Bảng 9. So sánh metrics chi tiết từ Artillery stress.log (trung bình 5 runs)

Kịch bản	Metric	PPO	WRR	Chênh lệch	Winner
Golden hour	P99 Latency (ms)	6,255.52	6,637.83	-5.80%	PPO
	Jitter (ms)	4,725.55	4,868.37	-2.90%	PPO
	Failure Rate (%)	89.36	88.57	+0.9%	WRR
	Throughput (reqs)	15,386	16,697	-7.90%	WRR
Video conference	P99 Latency (ms)	6,520.54	7,132.59	-8.60%	PPO
	Jitter (ms)	5,272.61	5,074.37	+3.9%	WRR
	Failure Rate (%)	81.76	81.55	+0.3%	WRR
	Throughput (reqs)	13,925	14,184	-1.80%	WRR
Low rate dos	P99 Latency (ms)	7,681.57	7,113.76	+8.0%	WRR
	Jitter (ms)	4,817.66	4,943.18	-2.50%	PPO
	Failure Rate (%)	78.54	77.64	+1.2%	WRR
	Throughput (reqs)	8,672	9,219	-5.90%	WRR
Hardware degradation	P99 Latency (ms)	7,488.86	6,443.34	+16.2%	WRR
	Jitter (ms)	5,283.67	4,783.19	+10.5%	WRR
	Failure Rate (%)	78.98	83.22	-5.10%	PPO
	Throughput (reqs)	17,675	13,926	+26.9%	PPO

Ghi chú: Jitter được tính từ standard deviation của p99 latency giữa các phases, Throughput được tính từ http.codes.200. Throughput được ghi trong bảng 1 để đo lường Application-level metrics (successful HTTP requests từ Artillery)

Bảng 10. Tổng hợp kết quả (trung bình 4 kịch bản)

Metric	PPO (avg)	WRR (avg)	Chênh lệch	Xu hướng
P99 Latency (ms)	6,986.62	6,831.63	+2.3%	WRR tốt hơn
Jitter (ms)	5,024.87	4,917.28	+2.2%	WRR tốt hơn
Failure Rate (%)	82.16	82.75	-0.7%	PPO tốt hơn
Throughput (reqs)	13,914.5	13,506.5	+3.0%	PPO tốt hơn

Phân tích: PPO có throughput cao hơn 3.0% nhưng Failure Rate thấp hơn 0.7%. WRR có latency và jitter tốt hơn. Điều này cho thấy sự đánh đổi giữa hiệu năng và độ ổn định.

D. Phân Tích Kết Quả

Kết quả thực nghiệm cho thấy sự phân hóa rõ rệt về hiệu năng giữa hai phương pháp tiếp cận trong các kịch bản vận hành khác nhau. Đáng chú ý, tác tử PPO thể hiện ưu thế vượt trội trong các kịch bản hệ thống phát sinh lỗi bất thường, ghi nhận mức tăng 8,6% về thông lượng trong điều kiện suy thoái phần cứng (Hardware Degradation).

Đặc biệt, phân tích sâu vào mức độ sử dụng tài nguyên (Resource Utilization) cho thấy năng lực phân bổ tài xuất sắc của thuật toán AI. Dựa trên dữ liệu OpenFlow, chỉ số công bằng Jain (Jain's Fairness Index tính trên tỷ lệ bytes/capacity) của PPO đạt mức 0.9650 – tiệm cận mức cân bằng lý tưởng. Tác tử đã tự động học được cách nhận diện năng lực máy chủ, san phẳng mức độ chịu tải giữa các node để tránh thất cổ chai. Ngược lại, thuật toán định tuyến tĩnh WRR bộc lộ điểm yếu chí mạng khi mùa quá tải lượng lớn lưu lượng vào server suy thoái, khiến chỉ số công bằng tụt dốc không phanh xuống mức 0.5174 (thấp hơn 86,5% so với PPO).

Tuy nhiên, trong điều kiện vận hành bình thường với lưu lượng đồng đều, WRR vẫn giữ vững vai trò là một baseline hiệu quả khi đạt thông lượng cao hơn từ 14,7% đến 18,6% so với PPO. Sự chênh lệch này xuất phát từ chi phí suy diễn (Inference overhead) của mạng Neural, khiến độ trễ đuôi P99 của PPO cao hơn 16,2% (7.488 ms so với 6.443 ms của WRR). Đây chính là sự đánh đổi tất yếu giữa tính thông minh và hiệu năng thô của hệ thống: việc chấp nhận giảm một phần thông lượng trong điều kiện lý tưởng là cái giá cần thiết để đổi lấy khả năng "tự chữa lành" (Self-healing) và duy trì tính sẵn sàng cao khi hạ tầng gặp sự cố.

Từ những phân tích trên, nghiên cứu khẳng định vai trò chiến lược của PPO không phải là thay thế hoàn toàn các thuật toán truyền thống, mà là hoạt động như một SLA Protector. Theo đó, hệ thống sẽ vận hành song song và chỉ kích hoạt cơ chế can thiệp thông minh của PPO khi các chỉ số giám sát mạng 20 chiều ghi nhận dấu hiệu bất thường hoặc vi phạm ngưỡng an toàn. Mô hình kiến trúc lại này đảm bảo tối ưu hóa đồng thời cả hai mục tiêu: duy trì độ trễ cực thấp trong trạng thái bình thường và bảo vệ nghiêm ngặt cam kết SLA uptime trong các tình huống cực đoan.

E. Hạn chế

Mặc dù đạt được những kết quả khả quan trong việc tối ưu hóa luồng, nghiên cứu vẫn tồn tại một số hạn chế nhất

định liên quan đến tính ổn định thống kê và quy mô triển khai. Trước hết, kết quả thực nghiệm ghi nhận mức độ biến thiên (variance) tương đối cao trong các kịch bản lưu lượng phức tạp như golden_hour (dao động từ -34,3% đến -7,9%) và video_conference (từ -26,7% đến +14,6). Điều này gợi ý rằng kích thước mẫu $n=5$ lượt chạy có thể chưa phản ánh hoàn toàn bản chất ngẫu nhiên của môi trường; do đó, các nghiên cứu tiếp theo cần mở rộng quy mô thực nghiệm với $n \geq 10$ để đạt được khoảng tin cậy 95% ổn định hơn. Phân tích sâu vào khoảng tin cậy 95% ($t=2.776, df=4$) cho thấy sự phân hóa rõ rệt về hiệu năng: trong khi các kịch bản golden_hour, video_conference và hardware_degradation cho thấy sự khác biệt có ý nghĩa thống kê giữa PPO và WRR (không có sự chồng lấn giữa các dải P99), thì kịch bản low_rate dos lại xuất hiện sự chồng lấn nhẹ giữa $P99_{PPO}$ [7.445-7.913] ms và $P99_{WRR}$ [6.708-7.539] ms. Điều này chứng minh rằng ưu thế của AI trong môi trường tấn công mạng vẫn cần được kiểm chứng thêm thông qua các hàm phần thưởng (reward functions) chuyên biệt hơn. Về mặt kiến trúc, mô hình hiện tại mới chỉ được đánh giá trên quy mô nhỏ với 3 máy chủ backend, điều này đặt ra thách thức về khả năng mở rộng (scalability) khi số lượng thực thể mạng tăng lên 6 hoặc 9 servers. Đồng thời, việc huấn luyện dựa hoàn toàn trên môi trường mô phỏng Gymnasium có thể dẫn đến khoảng cách giữa mô phỏng và thực tế (sim-to-real gap). Cuối cùng, chi phí suy diễn khiến độ trễ P99 của PPO cao hơn 16,2% so với thuật toán tính vẫn là một sự đánh đổi về mặt kỹ thuật. Đây chính là chi phí tính toán — cái giá tất yếu để hệ thống có được khả năng tự thích nghi khi đối mặt với các tình huống suy thoái hạ tầng cục đoạn, mở ra hướng phát triển cho các kỹ thuật tối ưu hóa trọng số mô hình hoặc triển khai domain randomization trong tương lai.

IV. KẾT LUẬN

Nghiên cứu đã thực hiện thành công việc tích hợp mô hình TFT-PPO vào bộ điều khiển SDN Ryu, đồng thời cung cấp các bằng chứng thực nghiệm quan trọng về khả năng ứng dụng của trí tuệ nhân tạo trong vai trò SLA Protector. Qua các kịch bản thử nghiệm trên môi trường Mininet/Ryu, kết quả cho thấy tác từ PPO thể hiện sự vượt trội rõ rệt trong các tình huống hệ thống phát sinh lỗi bất thường, cụ thể là mức tăng 8,6% thông lượng trong điều kiện suy thoái phần cứng. Ngược lại, đối với các kịch bản lưu lượng đồng đều, thuật toán tính như WRR vẫn chứng minh được tính hiệu quả và sự đơn giản khi duy trì mức thông lượng cao hơn từ 14,7% đến 18,6% so với mô hình AI. Những phát hiện này khẳng định rằng mô hình vận hành tối ưu nhất hiện nay là cấu trúc điều khiển lai (Hybrid Controller), trong đó PPO đóng vai trò

là lớp bảo vệ thông minh, chỉ kích hoạt khi hệ thống xuất hiện các dấu hiệu bất thường hoặc vi phạm ngưỡng sử dụng tài nguyên tối hạn.

Hướng phát triển tiếp theo của nghiên cứu sẽ tập trung vào việc tối ưu hóa hiệu năng và độ tin cậy của hệ thống thông qua kỹ thuật Knowledge Distillation nhằm nén mô hình PPO, giúp giảm độ trễ P99 xuống mức tương đương với các thuật toán tính mà vẫn giữ nguyên khả năng thích nghi linh hoạt. Bên cạnh đó, việc áp dụng Curriculum Learning để huấn luyện tác từ qua các kịch bản có độ phức tạp tăng dần và tích hợp cơ chế trí tuệ nhân tạo có thể giải thích (XAI) sẽ là những bước đi chiến lược. Những cải tiến này không chỉ giúp tăng cường độ chính xác cho quyết định điều phối mà còn xây dựng lòng tin cho các quản trị viên mạng khi triển khai các mô hình học tăng cường vào hạ tầng mạng SDN thực tế trong tương lai.

TÀI LIỆU THAM KHẢO

- [1] N. McKeown, T. Anderson, H. Balakrishnan, G. Parulkar, L. Peterson, J. Rexford, S. Shenker, and J. Turner, "OpenFlow: Enabling innovation in campus networks," ACM SIGCOMM Computer Communication Review, vol. 38, no. 2, pp. 69–74, Apr. 2008.
- [2] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," arXiv preprint arXiv:1707.06347, 2017.
- [3] B. Lim, S. Zohren, S. Roberts, and G. Wilson, "Temporal Fusion Transformers for interpretable multi-horizon time series forecasting," International Journal of Forecasting, vol. 37, no. 4, pp. 1748–1764, Oct. 2021.
- [4] R. Jain, D.-M. Chiu, and W. Hawe, "A Quantitative Measure of Fairness and Discrimination for Resource Allocation in Shared Computer Systems," Digital Equipment Corporation, DEC-TR-301, Sep. 1984.
- [5] Z. Wang, T. Schaul, M. Hessel, H. Hasselt, M. Lanctot, and N. de Freitas, "Dueling Network Architectures for Deep Reinforcement Learning," Proceedings of the 33rd International Conference on Machine Learning (ICML), pp. 1995–2003, 2016.
- [6] V. Mnih, A. P. Badia, M. Mirza, A. Graves, T. Lillicrap, T. Harley, D. Silver, and K. Kavukcuoglu, "Asynchronous Methods for Deep Reinforcement Learning," Proceedings of the 33rd International Conference on Machine Learning (ICML), pp. 1928–1937, 2016.
- [7] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.
- [8] B. Pfaff, J. Pettit, T. Koponen, E. J. Jackson, A. Zhou, J. Rajahalme, J. Gross, A. Wang, J. Stringer, P. Shelar, and N. McKeown, "The design and implementation of open vswitch," USENIX NSDI, pp. 117–130, 2015.
- [9] V. Sharma, G. Gupta, K. K. Singh, S. Chaudhary, and A. Singh, "Improving network resource allocation in SDN via deep learning-based load balancing," in Innovations in Computing, 1st ed. CRC Press, 2025, pp. 1-6.